VOLTV — Architecture Document

One document covering the full stack: languages, frameworks, every API endpoint, and which file on the frontend calls it.

---

1. Stack at a glance

Layer	Technology
Language (everywhere)	**TypeScript 5**
Frontend UI	**React 19.2** + **Next.js 16.2** App Router
Frontend bundler	**Turbopack** (`next dev --turbo`)
Styling	**Tailwind CSS v4** + Radix UI primitives
Icons / charts	Lucide-React, Recharts, Framer Motion
Forms & validation	React Hook Form + Zod
Client state	Zustand, TanStack Query
Backend runtime	Next.js **Route Handlers** (Node.js)
ORM	**Prisma 7.7** (`@prisma/adapter-pg`)
Database	**Neon Postgres** (serverless)
Cache / rate limit	**Redis** via `ioredis`
Auth	**Supabase Auth** (SSR) + custom admin cookie
External data	**TMDB v3** (`lib/tmdb.ts`)
Edge middleware	`proxy.ts` (auth gate + admin gate)
Deployment tunnel	Cloudflare Quick Tunnels (dev only)

There is **one language (TypeScript)** front and back. The "frontend" is React Server + Client Components in `app/`. The "backend" is Route Handlers also in `app/api/`. They share `lib/`, `utils/`, `types/`.

---

2. Frontend

Rendering model

- **Server Components** (default) — pages like `app/recommendations/page.tsx`, `app/profile/[username]/page.tsx`, `app/admin/page.tsx` read directly from Prisma at request time. No `fetch` required.
- **Client Components** (`"use client"`) — interactive pieces (modals, composers, toggles) that run in the browser and hit the Route Handlers over HTTP via `fetch`.

Key frontend directories

Path	Purpose
`app/`	Pages & route handlers (App Router)
`app/_components/`	Shared client components used across pages
`app/movie/[id]/_components/`	Movie-detail UI (rating modal, discussion tabs)
`app/admin/_components/`	Admin panel widgets (moderation, charts, debates)
`components/feed/`	Social feed cards, composer, reply modal
`components/compose/`	Full-screen post composer overlay
`components/movie/`	Rating modal, VOLTV meter, poster download
`components/profile/`	Follow button, edit profile, zoomable grids
`components/collections/`	Collection list, movie picker, edit/delete
`components/auth/`	Login modal
`components/layout/`	Topbar, Sidebar
`components/ui/`	Primitives (VerifiedBadge, etc.)
`hooks/`	`useAuth` , `useMovie` , `useStreak`

Design system

Tailwind utility classes, `clsx` + `tailwind-merge` for conditional styles, Radix UI for accessible dialogs/dropdowns/tooltips, Sonner for toasts, Framer Motion for transitions, Recharts for admin analytics.

---

3. Backend

Every file under `app/api/**/route.ts` is a backend endpoint. They export one or more of `GET`, `POST`, `PATCH`, `DELETE` functions. All use the same pattern:

1. `createServerSupabaseClient()` → verify the user's session cookie.
2. (Optional) Redis `rateLimit()` — e.g. ratings are 10/hour, votes are 5/min.
3. Zod `safeParse()` on the request body.
4. Prisma query against Neon Postgres.
5. Optional side-effects (awardXP, updateStreak, invalidate caches, activity feed).
6. `NextResponse.json(...)`.

Backend-only modules (`lib/`)

Module	What it does
`lib/prisma.ts`	Prisma client (singleton)
`lib/supabase-server.ts`	Supabase SSR client — reads auth cookie
`lib/tmdb.ts`	TMDB fetcher + trailer extractor
`lib/xp-system.ts`	`awardXP(userId, action)` + level thresholds
`lib/heatmap.ts`	Streak + watch-history heatmap generator
`lib/badge-engine.ts`	Evaluates and awards badges
`lib/recommendation-engine.ts`	5-layer hybrid recommender (main algorithm)
`lib/prediction-model.ts`	Rating prediction helper
`lib/comment-stats.ts`	Aggregated sentiment over movie comments
`lib/redis.ts`	`rateLimit`, `deleteCache`, `CacheKeys` helpers
`utils/voltv-score.ts`	5-dimension weighted score + `updateMovieScores()`

Auth / admin gating (`proxy.ts`)

Runs on every request. Three outcomes:

- `/admin/*` → requires `voltv_admin` cookie matching `ADMIN_COOKIE_SECRET`, else redirect `/admin/login`.
- `/api/poster`, `/admin/login`, `/api/admin/auth` → always pass through.
- Everything else → requires Supabase session (otherwise redirect `/login`).

---

4. Data layer

Neon Postgres (via Prisma 7)

Schema lives at `prisma/schema.prisma`. Key models:

- `User` — supabase_id, xp, level, phone_verified, is_admin, is_banned, is_pro
- `Movie` — tmdb_id (unique), genres[], voltv_score, tmdb_rating, trailer_url
- `Rating` — 5 dimensions (story/direction/acting/visuals/rewatch), overall_score, weight_applied, verdict — unique `(user_id, movie_id)`
- `Review` (used as "post") — content, media_urls, visibility, like_count, reply_count
- `Bookmark`, `ReviewLike`, `ReviewReply`, `Follow`
- `WatchHistory` — status enum, rewatch_count — unique `(user_id, movie_id)`
- `Collection`, `CollectionMovie`
- `DailyDebate` — movie_a/movie_b, votes_a/b, expires_at, **unique** `(debate_date, category)`
- `DebateVote` — unique `(user_id, debate_id)`
- `Notification`, `ActivityFeed`, `Prediction`, `Badge`, `Report`, `UserSimilarity`

Redis (ioredis)

Used for:

- Rate limiting (`rateLimit(userId, action, limit, windowSec)`)
- Caching hot reads (`CacheKeys.dailyDebate()`, `CacheKeys.heatmap(userId)`, `CacheKeys.movieTmdb(id)`)

---

5. External services

Service	Used via	Purpose
TMDB v3	`lib/tmdb.ts`	Movie metadata, trailers, search
Supabase	`lib/supabase-server.ts`	Auth (email/password + OAuth callbacks)
Neon	`DATABASE_URL` / `DIRECT_URL`	Primary Postgres
Redis	`lib/redis.ts`	Cache + rate limit
Cloudflare	`cloudflared` binary	Public URL for the dev server

---

6. API endpoint reference

Totals: 44 route files, 58 HTTP endpoints.

Each row lists: method, path, the route file, the frontend caller(s). Server Components that read data via Prisma directly (not via HTTP) are **not** listed as callers — they are noted where relevant.

6.1 Auth (5 endpoints)

Method	Path	Handler file	Frontend caller(s)
GET	`/api/auth/callback`	`app/api/auth/callback/route.ts`	Supabase OAuth redirect (no JS caller)
GET	`/api/auth/me`	`app/api/auth/me/route.ts`	(server-side only)
POST	`/api/auth/simple-login`	`app/api/auth/simple-login/route.ts`	`app/login/LoginForm.tsx:37`, `components/auth/LoginModal.tsx:38`
DELETE	`/api/auth/simple-login`	same	`components/layout/Topbar.tsx:113`
POST	`/api/auth/signout`	`app/api/auth/signout/route.ts`	`components/layout/Topbar.tsx:112`, `app/settings/page.tsx:53`

6.2 User & profile (10 endpoints)

Method	Path	Handler file	Frontend caller(s)
GET	`/api/user/me`	`app/api/user/me/route.ts`	`components/hooks/useMe.ts:25`
PATCH	`/api/user/profile`	`app/api/user/profile/route.ts`	`components/profile/EditProfileModal.tsx:68`
POST	`/api/user/upload`	`app/api/user/upload/route.ts`	`components/profile/EditProfileModal.tsx:48`, `components/feed/PostComposer.tsx:74`, `components/compose/ComposeOverlay.tsx:96`, `app/compose/ComposeModal.tsx:87`
GET	`/api/user/stats`	`app/api/user/stats/route.ts`	(server-side in profile page)
GET	`/api/user/heatmap`	`app/api/user/heatmap/route.ts`	(server-side in profile page)
GET	`/api/user/follow`	`app/api/user/follow/route.ts`	(server-side)
POST	`/api/user/follow`	same	`components/profile/FollowButton.tsx:25`
GET	`/api/user/compatibility`	`app/api/user/compatibility/route.ts`	(server-side in profile page)
GET	`/api/users/search`	`app/api/users/search/route.ts`	`app/search/page.tsx:92`
GET	`/api/streak/check`	`app/api/streak/check/route.ts`	`hooks/useStreak.ts:16`

6.3 Movies & ratings (6 endpoints)

Method	Path	Handler file	Frontend caller(s)
GET	`/api/movies/[id]`	`app/api/movies/[id]/route.ts`	`components/collections/AddMovieButton.tsx:60`, `app/collections/new/page.tsx:74`, `components/feed/PostComposer.tsx:96`, `components/compose/ComposeOverlay.tsx:117`, `app/compose/ComposeModal.tsx:108`
GET	`/api/movies/search`	`app/api/movies/search/route.ts`	`app/search/page.tsx:100`, `components/collections/AddMovieButton.tsx:40`, `app/collections/new/page.tsx:36`, `components/feed/PostComposer.tsx:53`, `components/compose/ComposeOverlay.tsx:68`, `app/compose/ComposeModal.tsx:59`
GET	`/api/movies/trending`	`app/api/movies/trending/route.ts`	(server-side in feed/discover)
GET	`/api/ratings`	`app/api/ratings/route.ts`	`hooks/useMovie.ts:21`

Method	Path	Handler file	Frontend caller(s)
POST	`/api/ratings`	same	`app/movie/[id]/_components/MovieActions.tsx:22`
GET	`/api/movie-comments`	`app/api/movie-comments/route.ts`	`app/movie/[id]/_components/CommentSection.tsx:64`, `app/movie/[id]/_components/ScoreMeter.tsx:26`, `app/movie/[id]/_components/ScoreGauge.tsx:18`
POST	`/api/movie-comments`	same	`app/movie/[id]/_components/CommentSection.tsx:79`

6.4 Watchlist (6 endpoints)

Method	Path	Handler file	Frontend caller(s)
GET	`/api/watchlist`	`app/api/watchlist/route.ts`	`hooks/useMovie.ts:22`
POST	`/api/watchlist`	same	`app/movie/[id]/_components/MovieActions.tsx:39`
DELETE	`/api/watchlist`	same	(client-side)
GET	`/api/watchlist/quick`	`app/api/watchlist/quick/route.ts`	`app/movie/[id]/_components/WatchActions.tsx:14`
POST	`/api/watchlist/quick`	same	`app/movie/[id]/_components/WatchActions.tsx:26`, `components/movie/PosterActions.tsx:28`
GET	`/api/watchlist/all`	`app/api/watchlist/all/route.ts`	`components/hooks/useWatchStatus.ts:24`

6.5 Reviews / posts (7 endpoints)

Method	Path	Handler file	Frontend caller(s)
GET	`/api/reviews`	`app/api/reviews/route.ts`	`app/movie/[id]/_components/DiscussionTabs.tsx:35`
POST	`/api/reviews`	same	`app/compose/ComposeModal.tsx:113`, `components/feed/PostComposer.tsx:102`, `components/compose/ComposeOverlay.tsx:122`, `app/movie/[id]/_components/DiscussionTabs.tsx:52`, `components/feed/ReplyModal.tsx:39`
PATCH	`/api/reviews/[id]`	`app/api/reviews/[id]/route.ts`	`components/feed/PostCard.tsx:148`
DELETE	`/api/reviews/[id]`	same	`components/feed/PostCard.tsx:177`
POST	`/api/reviews/[id]/like`	`app/api/reviews/[id]/like/route.ts`	`components/feed/PostCard.tsx:130`, `app/movie/[id]/_components/DiscussionTabs.tsx:192`
POST	`/api/reviews/[id]/bookmark`	`app/api/reviews/[id]/bookmark/route.ts`	`components/feed/PostCard.tsx:163`
POST	`/api/reviews/[id]/repost`	`app/api/reviews/[id]/repost/route.ts`	(client-side on PostCard)
POST	`/api/reviews/[id]/report`	`app/api/reviews/[id]/report/route.ts`	(client-side on PostCard menu)

6.6 Collections (6 endpoints)

Method	Path	Handler file	Frontend caller(s)
GET	`/api/collections`	`app/api/collections/route.ts`	(server-side)
POST	`/api/collections`	same	`app/collections/new/page.tsx:61`
PATCH	`/api/collections/[id]`	`app/api/collections/[id]/route.ts`	`components/collections/EditCollectionButton.tsx:36`
DELETE	`/api/collections/[id]`	same	`components/collections/EditCollectionButton.tsx:61`
POST	`/api/collections/[id]/movies`	`app/api/collections/[id]/movies/route.ts`	`app/collections/new/page.tsx:75`, `components/collections/AddMovieButton.tsx:62`
DELETE	`/api/collections/[id]/movies`	same	(client-side)

6.7 Debates (1 endpoint)

Method	Path	Handler file	Frontend caller(s)
POST	`/api/debates/vote`	`app/api/debates/vote/route.ts`	`app/_components/DebateClientWrapper.tsx:8`

6.8 Recommendations & predictions (3 endpoints)

Method	Path	Handler file	Frontend caller(s)
GET	`/api/recommendations`	`app/api/recommendations/route.ts`	(server-side in `/recommendations` page)
POST	`/api/recommendations/similarity`	`app/api/recommendations/similarity/route.ts`	(internal trigger)

Method	Path	Handler file	Frontend caller(s)
GET	`/api/predictions`	`app/api/predictions/route.ts`	(server-side)
POST	`/api/predictions`	same	(client-side on prediction UI)

6.9 Notifications (2 endpoints)

Method	Path	Handler file	Frontend caller(s)
GET	`/api/notifications`	`app/api/notifications/route.ts`	(server-side on `/notifications` page)
PATCH	`/api/notifications`	same	(client-side "mark all read")

6.10 Poster proxy (1 endpoint)

Method	Path	Handler file	Frontend caller(s)
GET	`/api/poster`	`app/api/poster/route.ts`	`components/movie/PosterDownloadButton.tsx` (built into an `<a href>` click)

Streams upstream image bytes with `Content-Disposition: attachment` so the browser downloads instead of displaying. Whitelisted in `proxy.ts` so it bypasses the auth gate.

6.11 Admin panel (11 endpoints)

Method	Path	Handler file	Frontend caller(s)
POST	`/api/admin/auth`	`app/api/admin/auth/route.ts`	`app/admin/login/AdminLoginForm.tsx:20`, `app/_components/LandingPage.tsx:127` (via `/api/admin/login`)
DELETE	`/api/admin/auth`	same	`app/admin/_components/AdminLogoutButton.tsx:15`
GET	`/api/admin`	`app/api/admin/route.ts`	(server-side on admin dashboard)
POST	`/api/admin`	same	`app/admin/_components/AdminReportCard.tsx:27`
POST	`/api/admin/users/[id]/admin`	`app/api/admin/users/[id]/admin/route.ts`	`app/admin/_components/AdminUserActions.tsx:34`
POST	`/api/admin/users/[id]/ban`	`app/api/admin/users/[id]/ban/route.ts`	`app/admin/_components/AdminUserActions.tsx:60`
POST	`/api/admin/posts/[id]/hide`	`app/api/admin/posts/[id]/hide/route.ts`	`app/admin/_components/AdminPostActions.tsx:29` (optimistic toggle)
GET	`/api/admin/movies/search`	`app/api/admin/movies/search/route.ts`	`app/admin/debates/_components/DebateForm.tsx:190`
POST	`/api/admin/debates`	`app/api/admin/debates/route.ts`	`app/admin/debates/_components/DebateForm.tsx:54`
DELETE	`/api/admin/debates/[id]`	`app/api/admin/debates/[id]/route.ts`	`app/admin/debates/_components/DeleteDebateButton.tsx:17`

---

7. Data flow examples

Rating a movie

1. User drags 5 sliders inside `components/movie/RatingModal.tsx` (opened from `MovieActions.tsx`).
2. Client POSTs `/api/ratings` with `{ movie_id, story, direction, acting, visuals, rewatch, voltv_verdict }`.
3. Route handler: Supabase auth → Redis rate limit (10/hr) → Zod validate → compute `overall_score` → Prisma upsert on `(user_id, movie_id)` → fire-and-forget: `updateMovieScores()`, `awardXP` (first time only — XP-farming guard), `updateStreak`, `checkAndAwardBadges`, `computeUserSimilarity`, invalidate heatmap cache.
4. Response includes `data` + `xp_gained`.

For-You recommendations

1. User visits `/recommendations` (Server Component `app/recommendations/page.tsx`).
2. Page calls `getCategorizedRecommendations(userId, 150)` from `lib/recommendation-engine.ts` — **Prisma directly, no HTTP**.
3. Engine pulls 400 candidates, scores with 5 layers (collaborative filtering via `UserSimilarity`, content-based cosine on taste DNA, ML feature scoring, deep-embedding cosine, recency boost), applies genre-diversity cap, then buckets into 9–12 Netflix-style rails (e.g. "Because you loved {top rated}", "Top Anime Picks", "Critically Acclaimed").
4. Server renders sections; no client fetch needed unless user refreshes.

Poster download

1. `PosterDownloadButton` creates `<a href="/api/poster?url=<tmdb-url>&name=<filename>">`.
2. `/api/poster` is whitelisted by `proxy.ts` — no auth required.

3. Handler verifies host is in allowlist (`image.tmdb.org` , `img.youtube.com` , `i.ytimg.com`), streams upstream bytes, sets `Content-Disposition: attachment` .

Admin hide/restore

1. Admin clicks eye icon. `AdminPostActions.tsx` flips optimistic state immediately.
2. `useRef<AbortController>` cancels any prior in-flight request (latest-click-wins).
3. POST `/api/admin/posts/[id]/hide` with `{ is_hidden }` .
4. On success → `router.refresh()` . On failure → revert optimistic state, toast error.

---

8. Project tree (top level)

```
voltv/
├── app/
│   ├── _components/              shared client components
│   ├── admin/                    admin panel (cookie-gated)
│   ├── api/                      44 route handlers — full list in §6
│   ├── collections/ compose/ debates/ discover/
│   ├── feed/ login/ movie/[id]/ notifications/
│   ├── profile/[username]/ recommendations/
│   ├── search/ settings/ social/
│   ├── layout.tsx  page.tsx  globals.css
├── components/                   reusable UI (feed, movie, profile, collections, ui)
├── hooks/                        useAuth, useMovie, useStreak
├── lib/                          prisma, supabase, tmdb, xp, heatmap,
│                                 recommendation-engine, badge-engine, redis
├── utils/                        voltv-score, formatters, personality
├── types/                        shared TS types
├── prisma/
│   ├── schema.prisma
│   └── migrations/
├── scripts/                      seed-debates, backfill-bluetick
├── proxy.ts                      auth + admin gate middleware
├── public/                       static assets
├── next.config.ts  tailwind.config.ts  tsconfig.json  package.json
└── README.md  AGENTS.md  ARCHITECTURE.md (this file)
```

---

9. Environment variables

```
DATABASE_URL=               # Neon pooled connection
DIRECT_URL=                 # Neon direct (used by Prisma migrations)
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=
TMDB_API_KEY=
REDIS_URL=                  # optional — defaults to in-memory fallback
ADMIN_USERNAME=
ADMIN_PASSWORD=
ADMIN_COOKIE_SECRET=        # signs the voltv_admin cookie
```

---

10. Summary numbers

- **Languages:** 1 (TypeScript)
- **Frameworks:** Next.js 16 (frontend + backend), Prisma 7 (DB), Supabase (auth)
- **Route files:** 44
- **HTTP endpoints:** 58 (sum of `GET/POST/PATCH/DELETE` exports across all routes)
- **Most-called endpoint:** `GET /api/movies/search` — 6 distinct callers (composers, search page, collection builders)
- **Top fan-in endpoint:** `POST /api/reviews` — 5 distinct callers (compose modal, post composer, overlay, reply modal, movie discussion)
- **Server-rendered pages hitting Prisma directly (no HTTP):** `/recommendations` , `/profile/[username]` , `/admin` , `/admin/debates` , `/debates` , `/notifications` , `/social` , `/leaderboard`